

Uczenie w grach i logika

Paweł Rychlikowski

Instytut Informatyki UWr

24 czerwca 2026

Co oblicza algorytm K -średnich? (dokończenie)

- Interesuje nas, aby **każdy element, jak najmniej się różnił od swojego reprezentanta** (czyli średniej, prototypu, centroidu)
- Miara: błąd średniokwadratowy, czyli

$$\sum_{x \in \text{Dane}} (x - \text{reprezentant}(x))^2$$

Definicja

Powyżej zdefiniowaną wielkość nazwiemy **błędem klasteryzacji**.

K-średnich jako minimalizacja błędu

Etap przypisywania

Prototypy ustalone. Każdy egzemplarz trafia do bliższego prototypu (czyli błąd maleje).

Etap liczenia średnich

Patrzmy na 1 klastę. Policzmy pochodną po c dla

$$\frac{1}{N} \sum_{i=1}^N (x_i - c)^2$$

Wychodzi:

$$\frac{2}{N} \sum_{i=1}^N (c - x_i)$$

Błąd klasteryzacji dla jednego klastra osiąga minimum w punkcie będącym średnią punktów klastra.

Wniosek

Oba etapy nie zwiększają błędu (tzn. jeżeli coś robią, to błąd maleje). Czyli osiągamy lokalne minimum.

K -średnich. Przykładowe zastosowania

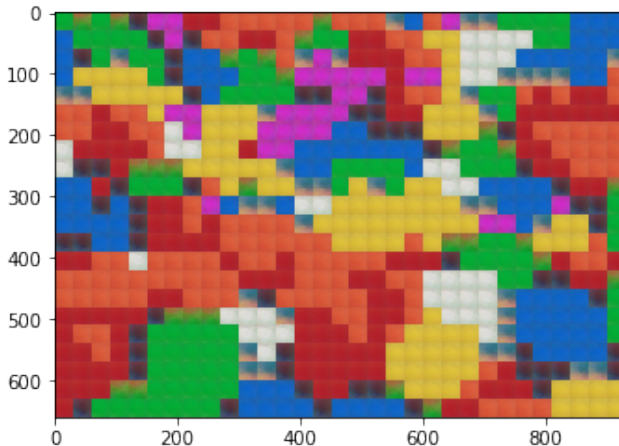
1. Wybór palety K kolorów, dostosowanej do obrazka.
2. Sklejanie obrazka z kwadratów (kompresja wektorowa)

K-średnik dla kolorów

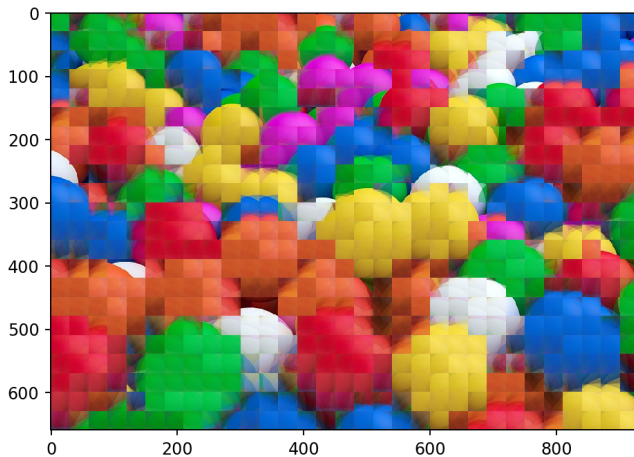
Popatrzymy, jak wybór palety i kompresja wektorowa działają dla przykładowego obrazka:



Na deser – wynik kompresji wektorowej, $K=10$



Na deser – wynik kompresji wektorowej, $K=100$



Uczenie funkcji oceniającej (w grach)

Idea

- Generuj dane z rozgrywek (historycznych, bądź wygenerowanych)
- Naucz się wag funkcji heurystycznej analizując te dane

Uwaga

W najprostszym przypadku (czyli liniowym) mamy:

$$V(s; w) = w \cdot \phi(s)$$

Definicja

Polityka odruchów (reflex policy) – to strategia agenta, w której podejmuje decyzje analizując przybliżoną funkcję oceniającą konsekwencje działań (w grach: stany po ruchu)

- Do generowania danych możemy wykorzystać politykę odruchową (czyli naszą aktualną funkcję oceniającą)
- **Problem:** tak wygenerujemy tylko jedną rozgrywkę (albo bardzo nie zróżnicowaną populację rozgrywek)

Generowanie danych (2)

Konieczne jest wprowadzenie losowości:

- a) Polityka ϵ -zachłanna (pamiętajmy o zmianie znaczenia V dla Mina i Maxa)
- b) Losowanie zgodne z prawdopodobieństwem „softmaxowym”, czyli:

$$P(s, a) = \frac{e^{V(\text{succ}(s,a))}}{\sum_{a' \in \text{Actions}(s)} e^{V(\text{succ}(s,a'))}}$$

- c) Dla gier z **z rzucaniem kostkami** (z elementem losowym) można wybierać zawsze optymalne ruchy (sama gra zapewnia czynnik eksploracyjny)

Uczenie funkcji oceniającej jako zadanie klasyfikacji/regresji

- **Klasyfikacja**: kto wygra (czy V jest dodatnie?)
- **Regresja**: wartość V

Idea

- Dla każdej wygenerowanej partii dokładnie wiemy, kto ją wygrał (powiedzmy, że MAX).
- Zatem wszystkie stany powinny wskazywać na przewagę MAXa (im późniejsze, tym bardziej)

Reguła TD-learningu

- **Predykcja:** $V(s; w)$
- **Cel:** $r + \gamma V(s'; w)$ (s' to kolejny stan w rozgrywce)

- Funkcja celu:

$$\frac{1}{2}(\text{prediction}(w) - \text{target})^2$$

- Gradient:

$$(\text{prediction}(w) - \text{target})\nabla_w(\text{prediction}(w))$$

- Reguła uaktualniania:

$$w \leftarrow w - \eta((\text{prediction}(w) - \text{target})\nabla_w(\text{prediction}(w)))$$

Algorytm

Dla każdego s, a, r, s' wykonuj:

$$w \leftarrow w - \eta(V(s; w) - (r + \gamma V(s', w))) \nabla_w V(s; w)$$

Dla funkcji liniowej:

$$V(s, w) = w \cdot \phi(s)$$

mamy

$$\nabla_w V(s, w) = \phi(s)$$

Porównanie TD-learning i Q-learning



Algorithm: TD learning

On each (s, a, r, s') :

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \underbrace{[\hat{V}_{\pi}(s; \mathbf{w})]}_{\text{prediction}} - \underbrace{(r + \gamma \hat{V}_{\pi}(s'; \mathbf{w}))}_{\text{target}} \nabla_{\mathbf{w}} \hat{V}_{\pi}(s; \mathbf{w})$$



Algorithm: Q-learning

On each (s, a, r, s') :

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \underbrace{[\hat{Q}_{\text{opt}}(s, a; \mathbf{w})]}_{\text{prediction}} - \underbrace{(r + \gamma \max_{a' \in \text{Actions}(s)} \hat{Q}_{\text{opt}}(s', a'; \mathbf{w}))}_{\text{target}} \nabla_{\mathbf{w}} \hat{Q}_{\text{opt}}(s, a; \mathbf{w})$$

- AlphaGo wygrało z Lee Sedolem (drugi gracz na świecie).

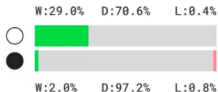
- **AlphaGo** wygrało z Lee Sedolem (drugi gracz na świecie). Zawierało elementy uczenia z historycznych partii – uczenie **z nadzorem**, jaki ruch zrobił dobry gracz, który w sytuacji X wygrał partię.
- **AlphaZero** – uczył się tylko grając sam ze sobą!
- Bardzo dobre (lub znakomite) wyniki w grach:
 - Go (mistrz Wszechświata)
 - Szachy (ale nie poziom mistrza świata)
 - Shoggi (jako przykład innej gry, która też działa)

AlphaZero

Chess



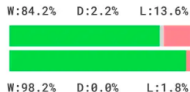
AlphaZero vs. Stockfish



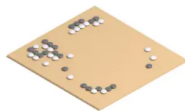
Shogi



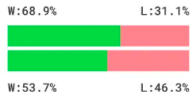
AlphaZero vs. Elmo



Go



AlphaZero vs. AGO

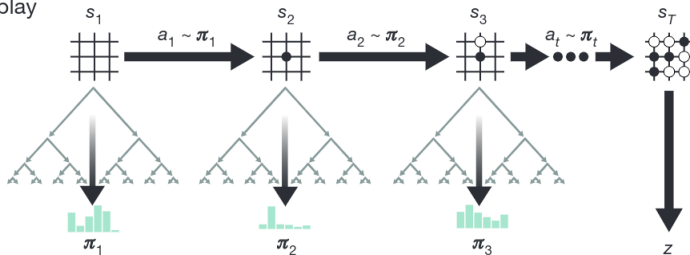


AZ wins  AZ draws  AZ loses  AZ white ○ AZ black ●

<https://deepmind.com/blog/article/alphazero-shedding-new-light-grand-games-chess-shogi-and-go>

AlphaZero. Rozgrywka

a Self-play



Drobna uwaga

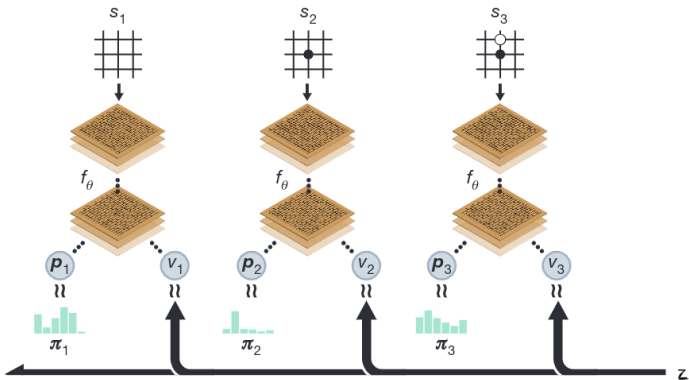
Trochę zmodyfikowany MCTS, który zamiast symulacji używa aktualnej funkcji heurystycznej.

AlphaZero. Analiza rozgrywki

Cele uczenia

- 1 Funkcja wybierająca ruch
- 2 Funkcja oceniająca planszę

b Neural network training



Zadanie

Stworzyć program, który będzie **odkrywał** ciekawe gry planszowe.

Dwa problemy do rozwiązania:

1. Jak opisać grę planszową?
2. Jak odróżnić fajną grę planszową od słabej? (ciekawsze pytanie)

Ewolucja gier planszowych

Co powinna uwzględniać funkcja oceny?

Kryteria używane w ocenie

- **Wewnętrzne** – jak wygląda opis gry (preferujemy poprawne, niezbyt długie i niezbyt skomplikowane)
- **Grywalność** – zbalansowana dla obu graczy, niezbyt dużo remisów, gry o satysfakcjonującej długości
- **Jakość rozgrywki**:
 - Dramatyzm: wyuczona funkcja oceniająca zmienia wartość w trakcie prawdziwych gier (najlepiej, żeby zmieniała się również przewaga)
 - Nieredukowalność: w rozegranych grach używane są wszystkie rodzaje ruchów
 - Krzywa uczenia: agent uczący się dłużej gra lepiej.

Oczywiście ważnym narzędziem jest TD-learning (żeby powstał jakiś sensowny agent, których grę możemy analizować).

Jak zarobić na ewolucji gier planszowych

- System **Ludi** służył do ewolucji gier planszowych.
- Podczas tygodnia ewolucji przeanalizowano **1389**, z czego **19** autorzy uznali za grywalne, a dwie, jak piszą **have proven to be of exceptional quality**
- Zapakowali je do pudełka i sprzedawali jako **pierwsze gry planszowe wymyślane przez maszynę**.

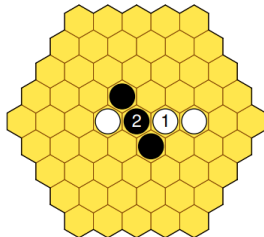
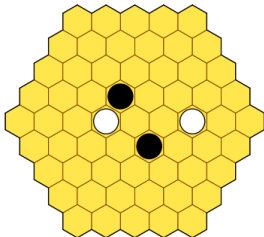
Można poczytać w pcgbook.com, rozdział 6

Gra Yavalath

Kod gry

```
(game Yavalath
  (players White Black)
  (board (tiling hex) (shape hex) (size 5))
  (end (All win (in-a-row 4)) (All lose (in-a-row 3))))
)
```

Przykładowa sytuacja:



- W 2022 roku pismo Communication of ACM wydrukowało artykuł pt. Reimagining Chess with AlphaZero
- Pytania:
 - Czy szachy można poprawić?
 - Czy inne wersje są ciekawsze?
 - Jak warianty gry zmieniają wartość bierek?

Podstawowa metoda

- 1 AlphaZero umie nauczyć się grać w dowolną grę, grając sam ze sobą.
- 2 Możemy zatem nauczyć go grać w **szachy z pięcioma skoczkami i słoniem** (lub w cokolwiek innego)
- 3 a następnie obserwować rozgrywki i wyciągać wnioski

AI is driving the next evolution of chess, giving players a glimpse into the game's future.

**BY NENAD TOMAŠEV, ULRICH PAQUET,
DEMIS HASSABIS, AND VLADIMIR KRAMNIK**

Reimagining Chess with AlphaZero

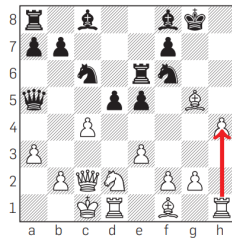
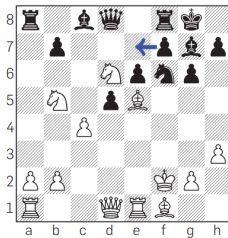
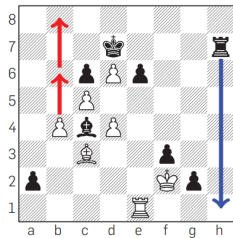
MODERN CHESS IS the culmination of centuries of experience, as well as an evolutionary sequence of rule adjustments from its inception in the 6th century to the modern rules we know today.¹⁷ While classical chess still captivates the minds of millions of players worldwide, the game is anything but static. Many variants have been proposed and played over the

game and is unlikely to ever fall out of fashion, alternative variants provide an avenue for more creative play. In Fischer random chess, the brainchild of former world champion Bobby Fischer, the initial position is randomized to counter the dominance of opening preparation in a game.⁷ One could consider not only entirely new ideas, but also reassess some of the newer additions to the game. For example, the “castling” move was only introduced in its current form in the 17th century. What would chess have been like had castling not been incorporated into the rules? Without recourse to repeating history, we reimagine chess and address such questions in silico with AlphaZero.²⁵

AlphaZero is a system that can learn superhuman chess strategies from scratch without any human supervision.^{19,22} It represents a milestone in artificial intelligence (AI), a field that has ventured down the corridors of chess more than once in search of challenges and inspiration. Throughout the history of computer chess, the focus was on creating systems that could spar with top human players over the board.³ Computer chess has progressed steadily since the 1950s, with better-tuned evaluation functions and enhanced search algorithms deployed on increasingly more computational

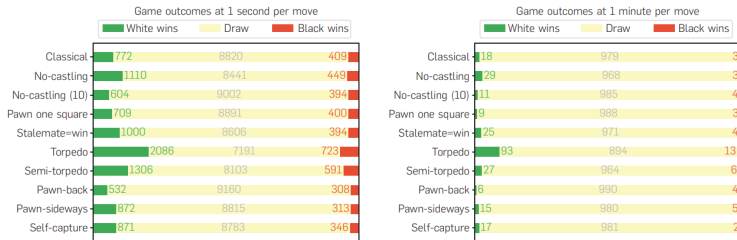
O szachach na nowo: przykładowe nowe ruchy

Figure 1. Examples by AlphaZero of three of the nine chess variants analyzed in this article. In Torpedo chess (left), White generates rapid counterplay with a torpedo move (b4-b6). ♖h1 is followed by yet another torpedo move, b6-b8=♖. In Pawn-sideways chess (center), Black plays a tactical sideways pawn move (f7-e7) after sacrificing a knight on f2 in the previous move, opening the f-file toward the White king. In Self-capture chess (right), White's self-capture move (♙xh4) generates threats against the Black king.



O szachach na nowo: rezultaty rozgrywek

Figure 2. AlphaZero self-play game outcomes: for 10,000 games played at 1 sec per move (left) and for 1,000 games played at 1 min per move (right).



O szachach na nowo: wartości bierek

Metoda

- 1 Przeprowadzać wiele rozgrywek szybkimi agentami, zapisywać przebieg
- 2 Uczyć liniowy klasyfikator przewidywać, czy sytuacja jest zwycięska na podstawie sumarycznej wartości bierek na planszy

Table 3. Estimated piece values from AlphaZero self-play games for each variant.

Variant					
Classical	1	3.05	3.33	5.63	9.5
No-castling	1	2.97	3.13	5.02	9.49
No-castling (10)	1	3.14	3.40	5.37	9.85
Pawn one square	1	2.95	3.14	5.36	9.62
Stalemate = win	1	2.95	3.13	4.76	8.96

Variant					
Self-capture	1	3.10	3.22	5.34	9.42
Pawn-back	1	2.65	2.85	4.67	9.39
Semi-torpedo	1	2.72	2.95	4.69	8.3
Torpedo	1	2.25	2.46	3.58	7.12
Pawn-sideways	1	1.8	1.98	2.99	5.92

Paradygmaty modelowania świata

- 1. Bazujące na **stanach**: przeszukiwanie, MDP, gry
- 2. Bazujące na **zmiennych**: CSP, sieci Bayesowskie (jeszcze przed nami)
- 3. Bazujące na **logice**: logika zdaniowa, logiki modalne, logika 1-go rzędu

Zajmiemy się teraz logiką. Zaczniemy od **logiki zdaniowej**.

- zmienne zdaniowe (przyjmują wartości 0/1)
- spójniki: $\vee$, $\wedge$, $\rightarrow$, $\leftrightarrow$, $\neg$

Przykłady

- $\text{pada} \wedge \neg \text{mam-parasol} \rightarrow \text{jestem-mokry} \vee \text{mam-kurtkę}$
- $\neg (p \vee q) \leftrightarrow (\neg p \wedge \neg q)$

Model (logika zdaniowa)

- **Modelem** w logice zdaniowej jest przypisanie zmiennym wartości logicznych.
 - Ogólnie o modelu myślimy jako o naszej wizji świata.
- **Interpretacją** formuły przy zadanym modelu jest zdefiniowana rekurencyjnie **wartość** formuły:
 - $I(a, w) = w(a)$, jeżeli a jest zmienną
 - $I(f_1 \vee f_2, w) = I(f_1, w) \vee I(f_2, w)$
 - (...) – inne, podobne reguły
- **Składnia (syntax)** vs **semantyka**

Definicja

Formuła f jest **spełnialna**, czyli ma model, jeżeli istnieje takie w , że $I(f, w) = 1$.

Uwaga

Takich przypisań jest skończenie wiele, stąd mamy prosty (wykładniczy) algorytm sprawdzania, czy formuła ma model (**jest spełnialna**)

- Formuły to zdania opisujące świat.
- Naturalne jest myślenie o zbiorze takich formuł (do którego możemy dodawać nowe fakty).
- Taki zbiór często nazywamy **bazą wiedzy**.

Koniunkcyjna postać normalna

Definicje

1. **literał** - **zmienna** albo $\neg$ **zmienna**
2. **klauzula** - $l_1 \vee \dots \vee l_n$ (gdzie l_i to literał)
3. **formuła w CNF** - $c_1 \wedge \dots \wedge c_n$, gdzie c_i jest klauzulą

Dlaczego CNF jest fajna?

- Każdą formułę można przekształcić do CNF (czasem płacąc wykładniczym wzrostem jej długości, ćwiczenia)
- Koniunkcja klauzul = zbiór klauzul = baza wiedzy
- Jak mamy zbiór formuł (bazę wiedzy, niekoniecznie w CNF) to wykładniczość dotyczy pojedynczej formuły, a nie całej bazy.

- Sprawdzanie spełnialności formuły boolowskiej jest zadaniem rozwiązywania więzów (baza wiedzy = zbiór więzów)
- Nie dziwi zatem, że podstawowe algorytmy (stosowane w praktyce) są dość podobne (**backtracking** + **propagacja**).

Uwaga

Współczesne **SAT-solvery** radzą sobie z milionami klauzul i setkami tysięcy zmiennych

A NP-zupełność?

- Oczywiście problem CNF-SAT (spełnialności formuły w CNF) jest **NP-zupełny**.
- Nie spodziewamy się istnienia algorytmu wielomianowego (znane algorytmy mają pesymistyczny czas wykładniczy).

Pytanie

Dlaczego SAT-Solvery działają dobrze?

Pytanie jest trudne, i tak do końca nie ma odpowiedzi. Jedyne, co można powiedzieć, że widocznie znaczna część w praktyce spotykanych formuł jest w jakimś sensie **łatwa**.

- Algorytm **Davisa–Putnama–Logemanna–Lovelanda (DPLL)** jest algorytmem znajdującym spełniające podstawienie dla formuły CNF.
- Jest **zupełny** – tzn. zawsze kończy się z prawidłowym wynikiem, może działać długo

Uwaga

Stanowi bazę współczesnych SAT-Solverów (z jednym usprawnieniem, o którym jeszcze powiemy).

Definicje

- a) **Klauzula jednostkowa** (unit clause) – klauzula zawierająca 1 literał
- b) **Czysty literał** – literał, który występuje tylko jako pozytywny, lub tylko jako negatywny (czyli z jedną polaryzacją).

$$x_1 \wedge \neg x_2 \wedge (x_3 \vee x_4 \vee x_5) \wedge (\neg x_3 \vee \neg x_4 \vee x_5)$$

Jakie wnioskowanie można przeprowadzić korzystając z tych pojęć:

- a) **unit propagation** – klauzule jednostkowe można spełnić na 1 sposób (spełniając literał), wstawiając wartość logiczną do innych klauzul możemy zrobić nowe klauzule jednostkowe.
- b) „Opłaca się” przypisywać **czystym literałom** wartość **true** (bo?).

Algorytm

Funkcja **DPLL**(Φ):

- jeśli Φ zawiera pustą klauzulę zwróć **false**
- dla każdej klazuli jednostkowej wykonaj **unit propagation** zmieniając Φ (do nasycenia)
- ustal wartości dla czystych literałów (zmieniając Φ)
- wybierz zmienną x (o nieokreślonej do tej pory wartości)
- zwróć **DPLL**($\Phi \wedge x$) **or** **DPLL**($\Phi \wedge \neg x$)

Oczywiście czasem wystarczy sprawdzić tylko jedną część rekurencyjnego wywołania!

- Zauważmy, że jak w algorytmie DPLL osiągnęliśmy sprzeczność, to można myśleć, że „udowodniliśmy” twierdzenie (przy założeniu analizowanej formuły Φ):

$$(l_1 \wedge l_2 \wedge \dots \wedge l_n) \rightarrow \text{Fałsz}$$

gdzie l_i jest literałem użytym w i -tym momencie algorytmu (w propagacji lub w backtrackingu)

- Możemy zatem uznać, że udowodniliśmy twierdzenie: ze zbioru klauzul wynika $\neg l_1 \vee \dots \vee \neg l_n$
- Trochę długa formuła (i nieużyteczna w propagacji), algorytm próbuje zatem znaleźć możliwie krótki ciąg literałów o ustalonych wartościach, który sam z siebie implikuje Fałsz (i zapamiętać go na przyszłość)